

Universidade de Caxias do Sul
Área do Conhecimento de Ciências Exatas e Engenharias
Disciplina de Fundamentos de Sistemas Operacionais
Professor Daniel Luis Notari
Avaliação I – 22/05/2022

Aluno: _____

8,3 9,2 em

1,5 Questão 1. (1,5) Diferentes algoritmos de escalonamento de CPU possuem diferentes propriedades e a escolha de um determinado algoritmo pode favorecer uma classe dos processos em detrimento de outra. Explique como funciona o algoritmo de Prioridades com Múltiplas filas com realimentação. Desenhe um exemplo.

1,0 Questão 2. (1,5) Um processo passa por uma série de estados de processo distinto. Em relação aos estados de transição de processos, explique o funcionamento das transições de estado:

- a) Quando um processo é despachado, ele transita de "pronto" para "em execução".
- b) Quando um processo é bloqueado, ele transita de "em execução" para "bloqueado".
- c) Quando um processo é considerado "zumbi" no Linux

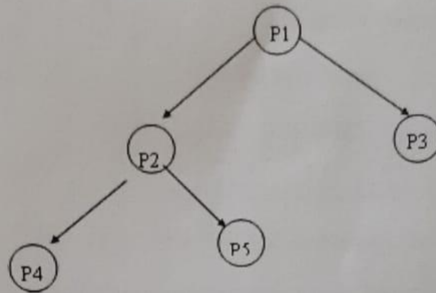
1,5 Questão 3. (1,5) Os Sistemas Operacionais líderes de mercados são Windows e Linux. Cite e explique os tipos de estrutura de sistemas operacionais que eles usam.

1,2 Questão 4. (1,5) Considere o conjunto de 4 processos abaixo, onde a chegada corresponde ao momento em que o processo foi posto a primeira vez no estado de apto e, o tempo de CPU é a quantidade de processamento necessário. Assumindo que a execução inicia no tempo 0 e, que a troca de contexto seja feita instantaneamente, ou seja, com um custo zero, determine o tempo médio de espera para uma política FCFS e Round Robin. OBS: considerar um quantum de 5 unidades.

Processo	Tempo de CPU
P0	15
P1	25
P2	20
P3	15

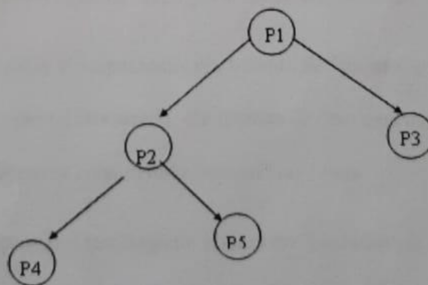
2,0

Questão 5. (2,0) Escreva um código correspondente ao grafo de hierarquia de processos abaixo utilizando a chamada de sistemas fork.



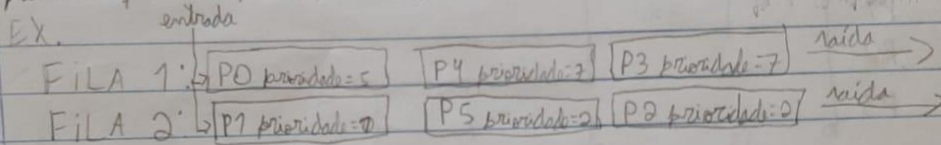
1,5

Questão 6. (2,0) Escreva um código correspondente ao grafo de hierarquia de processos abaixo utilizando a chamada de sistemas de pthreads com as primitivas de exclusão mútua.



Eduardo Eliehard Pereira

1) O algoritmo de prioridades com múltiplas filas com realimentação utiliza de um ordenamento baseado em diferentes níveis de prioridade, permitindo que o executor promova os processos mais importantes. Essa variação utiliza de múltiplas filas distintas para evitar starvation, que é quando um processo de baixa prioridade não consegue ser executado por sempre existir um processo de maior prioridade que ele. Com múltiplas filas, até os processos com menor prioridade recebem tempo de CPU. A realimentação faz com que as filas sejam reorganizadas baseado nos processos que as compõem, processando uma entrada de novos processos e uma diminuição das prioridades.



a) Quando o escalonador define que um processo na fila de prontos deve ser executado, ele realiza a troca de processador, salvando o estado do atual e recuperando o estado do selecionado. Assim, o processo que estava pronto agora está executando.

b) Quando um processo está esperando e tira o tempo de outro, esperando uma entrada de algum dispositivo de I/O (input/output) ou esperando algum outro dado, sua execução é interrombida e o processo é considerado bloqueado até receber o dado que espera.

c) Um processo é considerado guloso quando ele provoca deadlock no escalonador. Isto é, o escalonador não consegue decidir como proceder pois os processos estão, em geral, esperando um ao outro. Como ambos esperam um ao outro, o espera nunca termina e os processos ficam como gulosos, esperando para sempre.

3) As estruturas e arquiteturas dos sistemas operacionais Windows e Linux

não são mesmas, e se baseiam em:

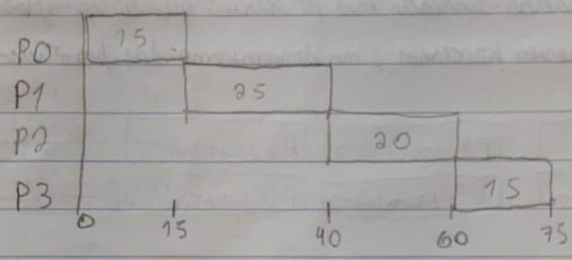
- monolítica: sem estrutura de módulos, qualquer rotina do SO pode chamar outra (ambos o usuário tenha restrições)

continua →

$$\begin{array}{r} 35 \\ -30 \\ \hline 5 \end{array}$$

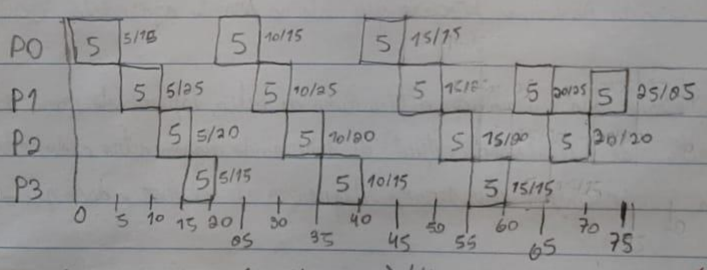
- em camadas: possui diferentes camadas com diferentes níveis de acesso, e faz uma requisição (tráfego) para acessar camadas mais internas.
- magnitude natural do sistema operacional é uma proteção em si, permitindo retrocompatibilidade.

4) FCFS:



Tempo de Espera: $(0+15+40+60)/4 = 28,75 \text{ u.t.}$ C

ROUND ROBIN: (quantum = 5)



Tempo de Espera: $(0+5+10+15)/4 = 6 \text{ u.t.}$

X

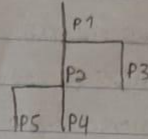
4

7.5

```

void main() {
5) pid_t pid-1, pid-2;
   P1(); ✓
   pid-1 = fork();
   if (pid-1 > 0) {
       P2(); ✓
       pid-2 = fork();
       if (pid-2 > 0) {
           P4(); ✓
       } else {
           P5(); ✓
       }
   } else {
       P3(); ✓
   }
}

```



```

int vector[5]; int P1=0, P2=1, P3=2, P4=3, P5=4;
6) pthread_mutex_t mutex;

```

```

void main() {
    pthread_t tid-1, tid-2, tid-3, tid-4, tid-5;
    pthread_mutex_init(&mutex, NULL);
    for(int i=0; i<5; i++) vector[i] = i+1;
    pthread_create(&tid-1, NULL, imprime-modo, (void*) P1);
    pthread_join(tid-1, NULL);
    pthread_create(&tid-2, NULL, imprime-modo, (void*) P2);
    pthread_create(&tid-3, NULL, imprime-modo, (void*) P3);
    pthread_join(tid-2, NULL);
    pthread_create(&tid-4, NULL, imprime-modo, (void*) P4);
    pthread_create(&tid-5, NULL, imprime-modo, (void*) P5);
    pthread_join(tid-3, NULL);
    pthread_join(tid-4, NULL);
    pthread_join(tid-5, NULL);
}

```

1 micron
esses
valores

(imprime modo na prax falha)
→